

DSA – LeetCode: 150 Days MAANG Roadmap

A Complete Guide

15 chapters compiled into one PDF

Snehal Kadiya

+91 9974031480

snehaliteng@gmail.com

<https://snehaliteng.github.io/>

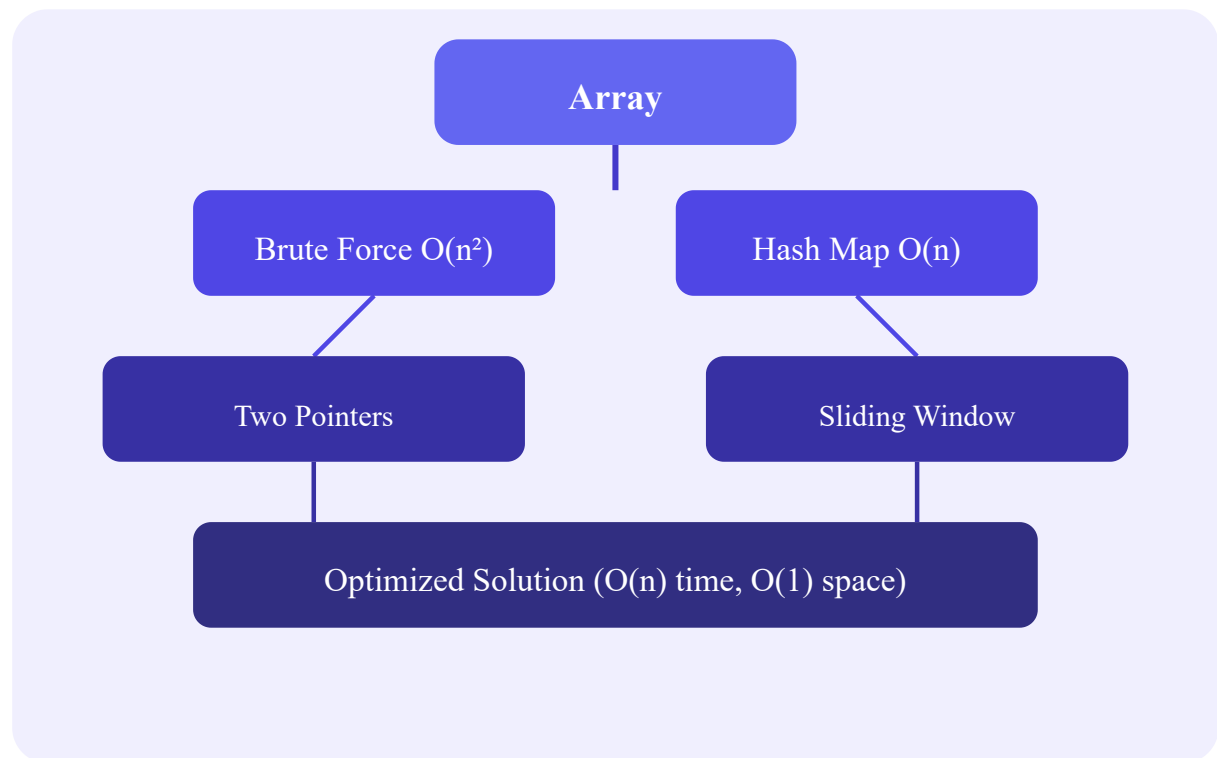
Copyright & Disclaimer

© 2026 Snehal Kadiya. All rights reserved.

This tutorial is intended for personal learning and educational purposes only. No part of this document may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the author.

Please do not print this document unnecessarily. Think of the environment — save paper and trees. Share the digital link instead:
<https://snehaliteng.github.io/tutorials/dsa-leetcode-roadmap/>

1. Arrays (Days 1–14)



Two Sum

```
# Brute Force  $O(n^2)$ 
def twoSum(nums, target):
    for i in range(len(nums)):
        for j in range(i+1, len(nums)):
            if nums[i] + nums[j] == target:
                return [i, j]

# Hash Map  $O(n)$ 
def twoSum(nums, target):
    seen = {}
    for i, val in enumerate(nums):
        diff = target - val
        if diff in seen:
            return [seen[diff], i]
    seen[val] = i
```

Best Time to Buy & Sell Stock

```
def maxProfit(prices):
    min_price = float('inf')
    max_profit = 0
    for price in prices:
        min_price = min(min_price, price)
        max_profit = max(max_profit, price - min_price)
    return max_profit
```

Majority Element

```
// C++ – Boyer-Moore Voting O(n)
int majorityElement(vector<int>& nums) {
    int count = 0, candidate = 0;
    for (int num : nums) {
        if (count == 0) candidate = num;
        count += (num == candidate) ? 1 : -1;
    }
    return candidate;
}
```

More Problems

Problem	Approach	Complexity
Move Zeroes	Two pointers, swap non-zero left	$O(n)$, $O(1)$
Squares of Sorted Array	Two pointers from ends	$O(n)$, $O(n)$
Merge Sorted Array	Merge from end	$O(m+n)$, $O(1)$
Pivot Index	Prefix sum	$O(n)$, $O(1)$
Running Sum	In-place prefix	$O(n)$, $O(1)$

 **Exercise:** Solve Two Sum in Python using a hash map, then implement the $O(1)$ space version of Move Zeroes. Aim for <15 min each.

[← Back to Tutorials](#)

2. Strings (Days 15–29)

Valid Palindrome

```
def isPalindrome(s: str) -> bool:
    l, r = 0, len(s) - 1
    while l < r:
        while l < r and not s[l].isalnum(): l += 1
        while l < r and not s[r].isalnum(): r -= 1
        if s[l].lower() != s[r].lower(): return False
        l += 1; r -= 1
    return True
```

Longest Common Prefix

```
// Java
public String longestCommonPrefix(String[] strs) {
    if (strs.length == 0) return "";
    String prefix = strs[0];
    for (int i = 1; i < strs.length; i++) {
        while (strs[i].indexOf(prefix) != 0) {
            prefix = prefix.substring(0, prefix.length() - 1);
        }
    }
    return prefix;
}
```

Valid Anagram

```
def isAnagram(s: str, t: str) -> bool:
    if len(s) != len(t): return False
    counts = [0] * 26
    for c in s: counts[ord(c) - 97] += 1
    for c in t: counts[ord(c) - 97] -= 1
    return all(c == 0 for c in counts)
```

More Problems

Problem	Pattern	Complexity
Reverse String	Two pointers, swap	$O(n)$, $O(1)$
Isomorphic Strings	Character mapping, bijection	$O(n)$, $O(1)$
Add Strings	Schoolbook addition, carry	$O(n)$, $O(n)$
FizzBuzz	Modulo, string building	$O(n)$, $O(1)$
Roman to Integer	Left-to-right, subtractive rule	$O(n)$, $O(1)$

 **Exercise:** Implement Valid Palindrome (ignore non-alphanumeric) and Longest Common Prefix. Write unit tests for edge cases (empty string, single char, all same).

[← Back to Tutorials](#)

3. Linked Lists (Days 30–44)

Reverse Linked List

```
def reverseList(head):
    prev, curr = None, head
    while curr:
        nxt = curr.next
        curr.next = prev
        prev = curr
        curr = nxt
    return prev
```

Merge Two Sorted Lists

```
def mergeTwoLists(l1, l2):
    dummy = ListNode()
    tail = dummy
    while l1 and l2:
        if l1.val < l2.val:
            tail.next = l1; l1 = l1.next
        else:
            tail.next = l2; l2 = l2.next
        tail = tail.next
    tail.next = l1 or l2
    return dummy.next
```

Linked List Cycle (Floyd's Algorithm)

```
// C++ - Tortoise & Hare
bool hasCycle(ListNode *head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}
```

More Problems

Problem	Pattern	Complexity
Palindrome Linked List	Find mid, reverse second half	$O(n)$, $O(1)$
Intersection of Two Lists	Two pointers, switch heads	$O(n)$, $O(1)$
Add Two Numbers	Dummy node, carry	$O(n)$, $O(n)$
Copy List with Random Pointer	Interleaved nodes, hash map	$O(n)$, $O(n)$

 **Exercise:** Implement Reverse Linked List (iterative + recursive), then solve Merge Two Sorted Lists. Use Floyd's algorithm to detect a cycle in a custom list.

[← Back to Tutorials](#)

4. Stacks & Queues (Days 45–59)

Valid Parentheses

```
def isValid(s: str) -> bool:
    stack = []
    pairs = {'(': ')', '[': ']', '{': '}'
    for c in s:
        if c in pairs:
            if not stack or stack.pop() != pairs[c]:
                return False
        else:
            stack.append(c)
    return not stack
```

Queue Using Stacks

```
// Java – Two stacks
class MyQueue {
    Stack<Integer> in = new Stack<>();
    Stack<Integer> out = new Stack<>();

    public void push(int x) { in.push(x); }
    public int pop() {
        if (out.isEmpty()) transfer();
        return out.pop();
    }
    public int peek() {
        if (out.isEmpty()) transfer();
        return out.peek();
    }
    private void transfer() {
        while (!in.isEmpty()) out.push(in.pop());
    }
}
```

Next Greater Element

```
def nextGreaterElement(nums):
    stack, result = [], [-1] * len(nums)
    for i in range(len(nums)):
        while stack and nums[i] > nums[stack[-1]]:
            result[stack.pop()] = nums[i]
        stack.append(i)
    return result
```

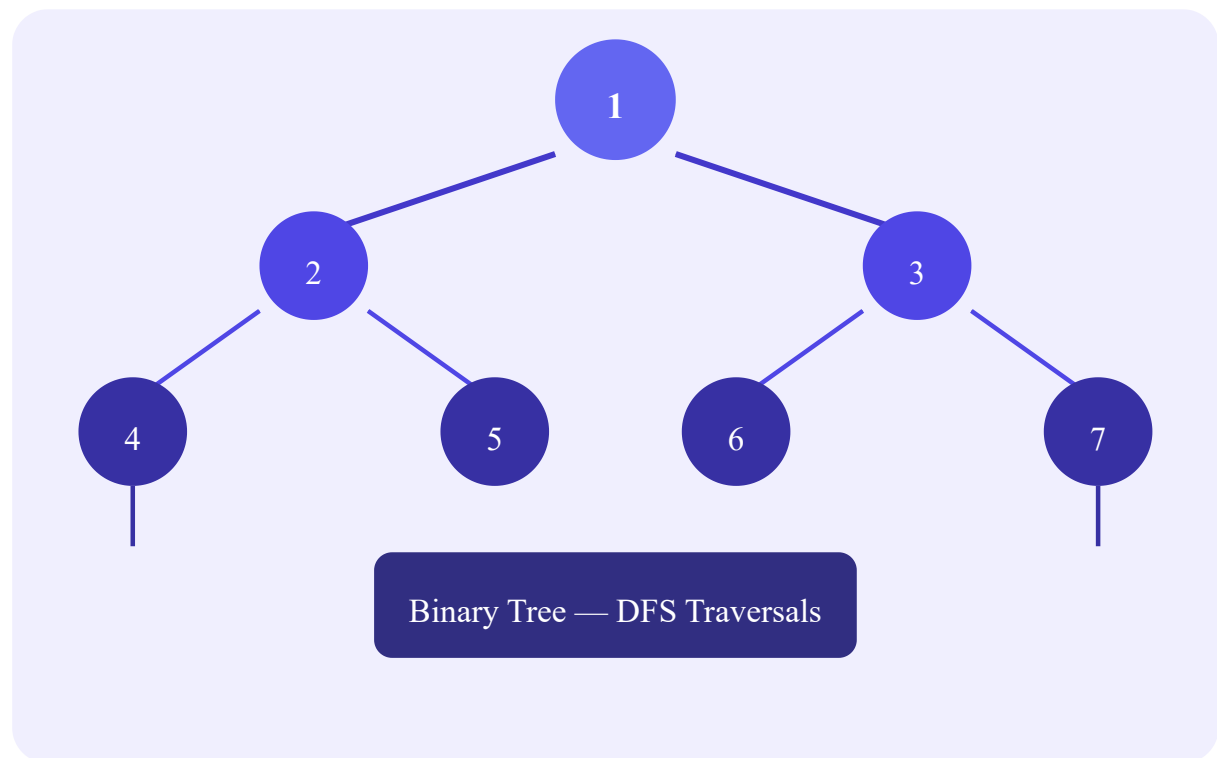
More Problems

Problem	Pattern	Complexity
Backspace String Compare	Stack simulation, two pointers	$O(n)$, $O(n)$
Min Stack	Two stacks (values + mins)	$O(1)$ per op
Decode String	Stack for counts and strings	$O(n)$, $O(n)$
Daily Temperatures	Monotonic decreasing stack	$O(n)$, $O(n)$

 **Exercise:** Implement Valid Parentheses with all bracket types, then build a Min Stack class with $O(1)$ `getMin()`. Solve Backspace Compare with $O(n)$ time.

[← Back to Tutorials](#)

5. Trees (Days 60–74)



Binary Tree Traversals

```
// Inorder (left → root → right)
def inorder(root):
    if not root: return
    inorder(root.left)
    print(root.val)
    inorder(root.right)

// Iterative inorder using stack
def inorder_iterative(root):
    stack, res = [], []
    curr = root
    while curr or stack:
        while curr:
            stack.append(curr)
            curr = curr.left
        curr = stack.pop()
        res.append(curr.val)
        curr = curr.right
    return res
```

Maximum Depth of Binary Tree

```
def maxDepth(root):
    if not root: return 0
    return 1 + max(maxDepth(root.left), maxDepth(root.right))
```

Lowest Common Ancestor (BST)

```
def lowestCommonAncestor(root, p, q):  
    while root:  
        if p.val < root.val and q.val < root.val:  
            root = root.left  
        elif p.val > root.val and q.val > root.val:  
            root = root.right  
        else:  
            return root
```

More Problems

Problem	Pattern	Complexity
Symmetric Tree	Recursive mirror check	$O(n)$, $O(h)$
Path Sum	DFS, target $\neq$ val	$O(n)$, $O(h)$
Serialize/Deserialize	Level-order or preorder + queue	$O(n)$, $O(n)$
BST Validate	Inorder traversal, check sorted	$O(n)$, $O(h)$

 **Exercise:** Implement recursive + iterative inorder traversal. Solve Maximum Depth and Symmetric Tree. Submit all three on LeetCode.

[← Back to Tutorials](#)

6. Binary Search (Days 75–89)

Binary Search Template

```
def binary_search(nums, target):
    l, r = 0, len(nums) - 1
    while l <= r:
        mid = l + (r - l) // 2
        if nums[mid] == target: return mid
        elif nums[mid] < target: l = mid + 1
        else: r = mid - 1
    return -1
```

Search Insert Position

```
// C++
int searchInsert(vector<int>& nums, int target) {
    int l = 0, r = nums.size();
    while (l < r) {
        int mid = l + (r - l) / 2;
        if (nums[mid] >= target) r = mid;
        else l = mid + 1;
    }
    return l;
}
```

First Bad Version

```
def firstBadVersion(n: int) -> int:
    l, r = 1, n
    while l < r:
        mid = l + (r - l) // 2
        if isBadVersion(mid): r = mid
        else: l = mid + 1
    return l
```

More Problems

Problem	Pattern	Complexity
Peak Element	Binary search on mountain	$O(\log n)$
Search in Rotated Sorted Array	Find pivot, then binary search	$O(\log n)$
Find Minimum in Rotated Array	Compare mid with right	$O(\log n)$
Median of Two Sorted Arrays	Partitioning, binary search on smaller	$O(\log(\min(m,n)))$

 **Exercise:** Implement Search Insert Position using the "lower bound" pattern. Solve First Bad Version. Aim for bug-free code in <20 min each.

[← Back to Tutorials](#)

7. Graphs (Days 90–104)

Number of Islands (DFS)

```
def numIslands(grid):
    if not grid: return 0
    count = 0
    m, n = len(grid), len(grid[0])

    def dfs(i, j):
        if i < 0 or i >= m or j < 0 or j >= n or grid[i][j] == '0':
            return
        grid[i][j] = '0'
        dfs(i+1, j); dfs(i-1, j)
        dfs(i, j+1); dfs(i, j-1)

    for i in range(m):
        for j in range(n):
            if grid[i][j] == '1':
                dfs(i, j)
                count += 1
    return count
```

Clone Graph (BFS)

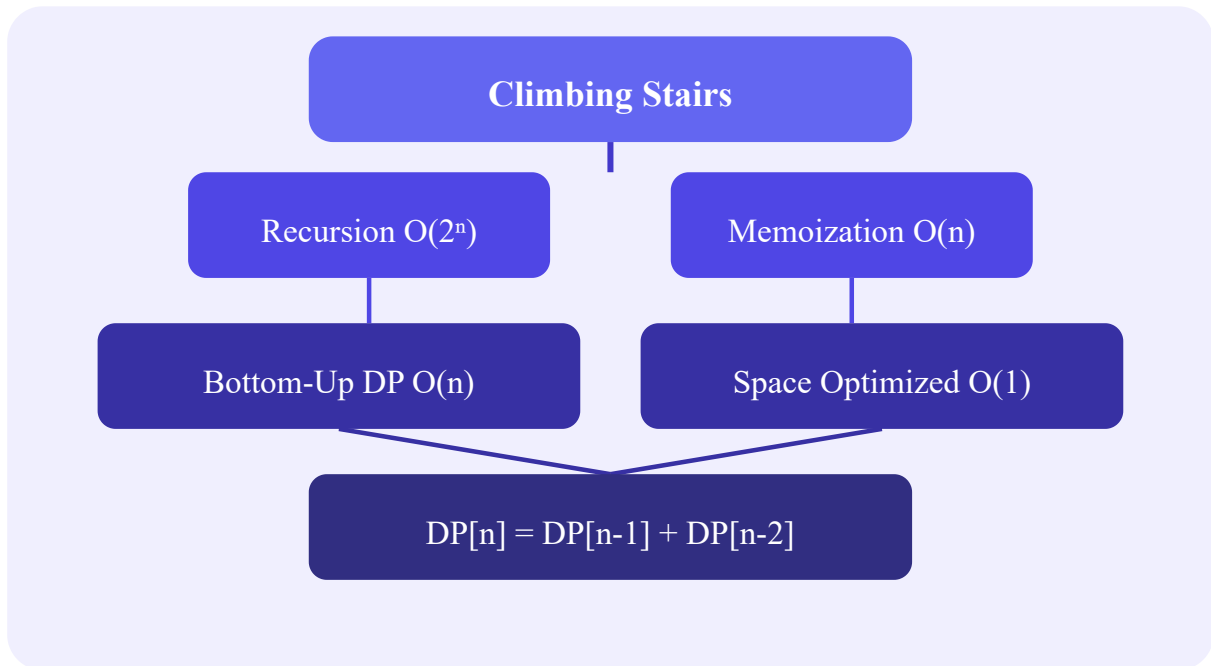
```
// Java - BFS with HashMap
public Node cloneGraph(Node node) {
    if (node == null) return null;
    Map<Node, Node> map = new HashMap<>();
    Queue<Node> q = new LinkedList<>();
    map.put(node, new Node(node.val));
    q.offer(node);
    while (!q.isEmpty()) {
        Node curr = q.poll();
        for (Node nei : curr.neighbors) {
            if (!map.containsKey(nei)) {
                map.put(nei, new Node(nei.val));
                q.offer(nei);
            }
            map.get(curr).neighbors.add(map.get(nei));
        }
    }
    return map.get(node);
}
```

More Problems

Problem	Pattern	Complexity
Flood Fill	DFS/BFS on grid	$O(mn)$
Island Perimeter	Count edges, subtract neighbors	$O(mn)$
Course Schedule	Kahn's algorithm (topological sort)	$O(V+E)$
Pacific Atlantic Water Flow	DFS from both oceans	$O(mn)$
Network Delay Time	Dijkstra's algorithm	$O(E \log V)$

 **Exercise:** Implement Number of Islands using DFS. Then solve Course Schedule (topological sort) using Kahn's algorithm. Run on sample test cases.

8. Dynamic Programming (Days 105–119)



Climbing Stairs (Fibonacci DP)

```
// O(n) time, O(1) space
def climbStairs(n):
    if n <= 2: return n
    a, b = 1, 2
    for _ in range(3, n+1):
        a, b = b, a + b
    return b
```

Coin Change (Min Coins)

```
def coinChange(coins, amount):
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0
    for coin in coins:
        for x in range(coin, amount + 1):
            dp[x] = min(dp[x], dp[x - coin] + 1)
    return dp[amount] if dp[amount] != float('inf') else -1
```

Longest Increasing Subsequence

```
// C++ - DP O(n²)
int lengthOfLIS(vector<int>& nums) {
    vector<int> dp(nums.size(), 1);
    int ans = 1;
    for (int i = 1; i < nums.size(); i++) {
        for (int j = 0; j < i; j++) {
            if (nums[j] < nums[i])
                dp[i] = max(dp[i], dp[j] + 1);
        }
        ans = max(ans, dp[i]);
    }
    return ans;
}
```

More Problems

Problem	Pattern	Complexity
House Robber	$DP[i] = \max(DP[i-1], DP[i-2] + \text{nums}[i])$	$O(n), O(1)$
Decode Ways	DP with single and double digit checks	$O(n), O(n)$
Jump Game	Greedy reachable index	$O(n), O(1)$
Container With Most Water	Two pointers, $\text{area} = \text{width} \times \min(h[l], h[r])$	$O(n), O(1)$
Subarray Sum Equals K	Prefix sum + hash map	$O(n), O(n)$
3Sum	Sort + two pointers	$O(n^2), O(1)$
Merge Intervals	Sort by start, merge overlapping	$O(n \log n), O(n)$
Pascal's Triangle	Bottom-up row generation	$O(n^2), O(n^2)$

 **Exercise:** Solve Climbing Stairs and Coin Change. Then implement Longest Increasing Subsequence in $O(n \log n)$ using patience sorting.

[← Back to Tutorials](#)

9. Greedy & Advanced (Days 120–134)

Interval Scheduling (Non-overlapping Intervals)

```
def eraseOverlapIntervals(intervals):
    intervals.sort(key=lambda x: x[1])
    end = float('-inf')
    count = 0
    for s, e in intervals:
        if s >= end:
            end = e
        else:
            count += 1
    return count
```

Gas Station

```
def canCompleteCircuit(gas, cost):
    if sum(gas) < sum(cost): return -1
    tank = start = 0
    for i in range(len(gas)):
        tank += gas[i] - cost[i]
        if tank < 0:
            start = i + 1
            tank = 0
    return start
```

Trapping Rain Water

```
// C++ – Two pointers
int trap(vector<int>& height) {
    int l = 0, r = height.size() - 1;
    int leftMax = 0, rightMax = 0, water = 0;
    while (l < r) {
        if (height[l] < height[r]) {
            if (height[l] >= leftMax) leftMax = height[l];
            else water += leftMax - height[l];
            l++;
        } else {
            if (height[r] >= rightMax) rightMax = height[r];
            else water += rightMax - height[r];
            r--;
        }
    }
    return water;
}
```

More Problems

Problem	Pattern	Complexity
Partition Labels	Last occurrence, greedy partition	$O(n)$, $O(1)$
Candy	Left-to-right, right-to-left passes	$O(n)$, $O(n)$
Skyline Problem	Max-heap, sweep line	$O(n \log n)$

 **Exercise:** Implement Interval Scheduling (non-overlapping), Gas Station, and Trapping Rain Water. Aim to solve all three within 45 min.

10. Backtracking (Days 135–145)

Backtracking Template

```
def backtrack(path, choices, result):
    if is_solution(path):
        result.append(path[:])    # shallow copy
        return
    for choice in choices:
        if is_valid(choice):
            path.append(choice)
            backtrack(path, choices, result)
            path.pop()            # undo
```

Permutations

```
def permute(nums):
    result = []
    def backtrack(path, remaining):
        if not remaining:
            result.append(path[:])
            return
        for i in range(len(remaining)):
            path.append(remaining[i])
            backtrack(path, remaining[:i] + remaining[i+1:])
            path.pop()
    backtrack([], nums)
    return result
```

Generate Parentheses

```
// Java
public List<String> generateParenthesis(int n) {
    List<String> res = new ArrayList<>();
    backtrack(res, "", 0, 0, n);
    return res;
}

void backtrack(List<String> res, String curr, int open, int close, int max)
    if (curr.length() == max * 2) { res.add(curr); return; }
    if (open < max) backtrack(res, curr + "(", open+1, close, max);
    if (close < open) backtrack(res, curr + ")", open, close+1, max);
}
```

More Problems

Problem	Pattern	Complexity
Combinations	Backtrack with start index	$O(C(n,k))$
N-Queens	Column + diagonal sets	$O(n!)$
Sudoku Solver	Row/col/box validation, backtrack	$O(9^{n^2})$
Word Search	DFS + visited set	$O(mn \cdot 4^L)$

 **Exercise:** Implement the general backtracking template. Solve Permutations and Generate Parentheses. Then attempt N-Queens on a 4×4 board.

11. System Design (Days 146–150)

Rate Limiter

Design a rate limiter (e.g., token bucket, sliding window log) to cap API requests per user per second.

```
# Token Bucket
class TokenBucket:
    def __init__(self, capacity, refill_rate):
        self.capacity = capacity
        self.tokens = capacity
        self.refill_rate = refill_rate
        self.last_refill = time.time()

    def allow_request(self):
        now = time.time()
        elapsed = now - self.last_refill
        self.tokens = min(self.capacity,
                          self.tokens + elapsed * self.refill_rate)
        self.last_refill = now
        if self.tokens >= 1:
            self.tokens -= 1
            return True
        return False
```

Parking Lot System

Design a multi-level parking lot: multiple floors, spot types (compact/large/motorcycle), ticketing, payment, availability tracking.

```
# Key classes: ParkingLot, Floor, Spot, Ticket, Vehicle
# Patterns: Singleton for ParkingLot, Factory for Spot types
# Data structures: Map<Floor, List<Spot>>, PriorityQueue for nearest
```

Design Problems Overview

Problem	Key Components	Patterns
Library Management	Books, members, check-in/out, fines	Repository, MVC
Amazon Shopping	Products, cart, orders, payments, inventory	Microservices, event-driven
Notification System	Email, SMS, push; templates, queue	Observer, pub-sub
Uber Backend	Rider/driver matching, pricing, ETA, trip history	QuadTree, pub-sub, gRPC
Google Drive	File upload/download, sync, sharing, versioning	Storage hierarchy, CRDT
Twitter	Tweets, timeline, follows, trends, search	Fan-out, cache (Redis)
YouTube	Video upload/stream, transcoding, CDN, recommendations	Async processing, CDN

 **Exercise:** Design a rate limiter using the sliding window log approach. Draw the architecture diagram. Write pseudocode for the `allow_request()` method.

[← Back to Tutorials](#)

12. Object-Oriented Design

OOD Design Template

```
// Step 1: Identify key entities (nouns)
// Step 2: Define relationships (inheritance, composition)
// Step 3: List behaviors (verbs → methods)
// Step 4: Apply design patterns (Singleton, Factory, Strategy, Observer)
// Step 5: Discuss trade-offs (coupling vs cohesion, extensibility)
```

ATM Machine

```
# Key classes

class Card:
    def __init__(self, number, pin, account):
        self.number = number; self.pin = pin
        self.account = account

class Account:
    def __init__(self, acct_no, balance=0):
        self.acct_no = acct_no; self.balance = balance

    def withdraw(self, amount):
        if amount > self.balance: raise InsufficientFunds()
        self.balance -= amount
        return amount

class ATM:
    def __init__(self, cash_available):
        self.cash_available = cash_available

    def authenticate(self, card, pin):
        return card.pin == pin

    def withdraw(self, card, amount):
        if amount > self.cash_available: raise ATMOutOfCash()
        cash = card.account.withdraw(amount)
        self.cash_available -= amount
        return cash
```

Key OOD Problems

Problem	Key Classes	Patterns
Airline Management	Flight, Seat, Passenger, Booking, Payment	Factory, Singleton
Restaurant System	Menu, Order, Chef, Waiter, Table	Command, Observer
Car Rental	Vehicle, Customer, Reservation, Invoice	Strategy (pricing)
Chess Game	Board, Piece (subclasses), Player, Move	Strategy, State
Ticketmaster	Event, Venue, Seat, Booking, Payment	Factory, Proxy
LinkedIn	Profile, Connection, Post, Message, Job	Observer, MVC
Facebook	User, Post, Comment, Like, Friend, Group	Observer, Composite

 **Exercise:** Design an ATM system with cash dispensing, deposit, and balance inquiry. Draw a class diagram showing relationships. Implement `withdraw()` in Python.

13. Behavioral Interview Prep

Common Questions & Frameworks

Question	Framework	Example
Tell me about yourself	Present → Past → Future	"I'm a software engineer at X. Previously at Y, I built Z. I want to work on..."
Strengths & Weaknesses	STAR for strengths; honest + improvement plan for weakness	"I excel at system design. My weakness is public speaking — I joined Toastmasters."
Ambiguity	Clarify → Break down → Iterate	"When requirements are unclear, I ask stakeholders, prototype, and validate."
Conflict resolution	Listen → Understand → Propose solution	"Two devs disagreed on architecture. I facilitated a meeting with data to decide."
Failure / Mistake	Own → Learn → Improve	"I merged untested code. Now I enforce CI checks."
Leadership	Influence without authority	"I mentored 3 juniors and established code review standards."

MAANG Leadership Principles (Amazon LP)

1. Customer Obsession
2. Ownership
3. Invent and Simplify
4. Are Right, A Lot
5. Learn and Be Curious
6. Hire and Develop the Best
7. Insist on the Highest Standards
8. Think Big
9. Bias for Action
10. Frugality
11. Earn Trust
12. Dive Deep
13. Have Backbone; Disagree and Commit
14. Deliver Results
15. Strive to be Earth's Best Employer
16. Success and Scale Bring Broad Responsibility

STAR Method

- **Situation** — Set the context
- **Task** — What needed to be done
- **Action** — What you specifically did
- **Result** — Outcome (quantified if possible)

 **Exercise:** Prepare STAR stories for: conflict resolution, a time you took initiative, a failure you learned from, and a time you influenced a technical decision. Practice aloud.

14. Revision Days — Checkpoints

Every 15 days, pause and revisit previously solved problems. Spaced repetition ensures long-term retention.

Checkpoint	Day	Topics to Revise	Suggested Problems
R1	14	Arrays, Strings	Two Sum, Valid Palindrome, Valid Anagram, Reverse String
R2	29	Linked Lists, Stacks & Queues	Reverse Linked List, Merge Two Lists, Valid Parentheses, Min Stack
R3	44	Trees, BST	Max Depth, Symmetric Tree, LCA, Inorder traversal
R4	59	Binary Search	Search Insert Position, Peak Element, Rotated Array
R5	74	Graphs	Number of Islands, Clone Graph, Course Schedule
R6	89	Dynamic Programming	Climbing Stairs, Coin Change, House Robber, LIS
R7	104	Greedy	Gas Station, Trapping Rain Water, Partition Labels
R8	119	Backtracking	Permutations, Generate Parentheses, N-Queens
R9	134	All DSA topics	Pick 5 hard problems across topics
R10	135	All DSA topics + SD	Timed mock — 3 problems in 90 min

Revision Strategy

- Re-solve each problem from scratch (no peeking at solution)

- Compare your approach with the optimal solution
- If you can't solve in 15 min, review the pattern and try again tomorrow
- Track accuracy in a spreadsheet — target >80% on revision day

 **Exercise:** On your next revision day, re-solve Two Sum, Reverse Linked List, Max Depth, Number of Islands, and Climbing Stairs without looking at solutions. Time yourself.

15. Bonus: Wrap-Up & Resources

Final Wrap-Up

Congratulations on completing the 150-day MAANG roadmap! Here's what you've mastered:

- **DSA:** Arrays, Strings, Linked Lists, Stacks, Queues, Trees, Graphs, DP, Greedy, Backtracking
- **System Design:** Rate Limiter, Parking Lot, Amazon Shopping, Uber, Twitter, YouTube, Google Drive
- **OOD:** ATM, Airline, Restaurant, Chess, Ticketmaster, LinkedIn, Facebook
- **Behavioral:** STAR stories, Leadership Principles, Conflict Resolution, Initiative

Next Steps

1. **Mock Interviews** — Practice on Pramp, Interviewing.io, or with a friend
2. **Company-Specific Prep** — Review recent problems from your target company on LeetCode Discuss
3. **Apply** — Referrals > direct applications. Use LinkedIn and Blind
4. **Negotiate** — Know your worth. Levels.fyi for compensation benchmarks

Quick Guide

Topic	Key Pattern	Common Mistakes
Arrays	Hash maps, two pointers	Off-by-one, edge cases (empty, single)
Strings	Sliding window, character maps	Case sensitivity, non-alphanumeric
Linked Lists	Dummy node, two pointers	Null pointer, cycle detection
Trees	Recursion, stack for iterative	Base case, depth limit
DP	Top-down (memo) vs bottom-up (tab)	Wrong recurrence relation
Graphs	BFS for shortest path, DFS for connectivity	Visited set, infinite loops
System Design	Estimate → Define API → Data model → Architecture → Scale	Missing non-functional reqs

Useful Resources

- **LeetCode** — Problem solving, discuss, mock interviews
- **NeetCode.io** — Pattern-based roadmap, videos
- **Grokking the Coding Interview** — Course for patterns
- **Designing Data-Intensive Applications** — Book for system design
- **Levels.fyi** — Salary benchmarks
- **Blind** — Anonymous career discussions

 **Exercise:** Schedule a mock interview this week. Review the quick guide and identify your weakest topic — spend the next 3 days strengthening it before the mock.